

# Task 02. MapReduce Matrix Multiplication & RecSys.

## Industrial Machine Learning on Hadoop and Spark, Fall 2025

Task start date: October 14, 2025, 23:59CET.  
Hard Deadline: **October 28, 2025, 23:59CET.**

### Assignment Description

This assignment is aimed at introducing the Apache Hadoop infrastructure.  
The assignment consists of two parts:

1. Implementing and running a simple Matrix Multiplication program using MapReduce.
2. Implementing and running a complex Collaborative filtering pipeline using MapReduce.

### 1 Matrix Multiplication (40%)

In this part, you need to implement a Hadoop Streaming program that performs matrix multiplication.  
Let  $A \in \mathbb{R}^{n \times m}$ ,  $B \in \mathbb{R}^{m \times k}$ ,  $C = AB \in \mathbb{R}^{n \times k}$ .

We will represent input data (i.e., matrices  $A$  and  $B$ ) as files `A.txt`, `B.txt`, where matrix elements are written line by line separated by spaces. Each row will be explicitly numbered. The result of the multiplication (`C.txt`) will be represented as key-value pairs, where the key is a pair (row number, column number). The order of rows will be ignored.

$$A = \begin{pmatrix} 3 & 4 & 3 & 2 \\ 0 & 5 & 2 & 1 \\ 0 & 4 & 2 & 0 \end{pmatrix} \Rightarrow \underbrace{\begin{matrix} 0 & 3 & 4 & 3 & 2 \\ 1 & 0 & 5 & 2 & 1 \\ 2 & 0 & 4 & 2 & 0 \end{matrix}}_{A.txt} \quad C = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \Rightarrow \underbrace{\begin{matrix} 0,1 & 2 \\ 0,0 & 1 \\ 1,0 & 3 \\ 1,1 & 4 \end{matrix}}_{C.txt}$$

You need to implement a MapReduce program that takes matrices  $A, B$  in the above format as input, computes their product, and outputs the result as a list of key-value pairs.

The assignment provides auxiliary scripts that simplify the task:

1. `generate_task.py` — generates random matrices  $A, B$  of the specified size, computes their product, and saves  $A, B, C$  in the required format.

Example run: `python3 generate_task.py -n 10 -m 20 -k 30`

2. `check_answer.py` — compares the true result `C.txt` with the one produced by the MapReduce program. The script accounts for the result being split into multiple files according to the number of reducers.

Example run: `python3 check_answer.py -n 10 -m 20 -k 30`

You must implement the following scripts:

1. `mapper.py`, `reducer.py` — code for the mapper and reducer for matrix multiplication
2. `run_hadoop.sh` — code for preparing directories on NameNode, running the Hadoop Streaming job, and copying the result from HDFS to the local filesystem
3. `run.sh` — script to run the entire pipeline. This script should accept the problem dimensions  $(n, m, k)$ , the number of mappers, and the number of reducers as command-line arguments.

First, the matrices  $A, B, C$  of the specified size are generated, then the data is copied into the container, and the Hadoop Streaming job is launched on NameNode. After the MapReduce job completes, the script copies the result from the container and checks its correctness.

Example run: `sh run.sh 10 20 30 5 5`

As a result, the project must exactly match the structure shown in Figure 3. Submit an archive of this structure named `<FullName>_task_02.1.zip`. The `data` folder may be omitted.

Full points will be awarded only if running `run.sh` with different parameters completes successfully. Your program must correctly handle problem dimensions  $n, m, k \leq 500$  for any number of mappers and reducers.

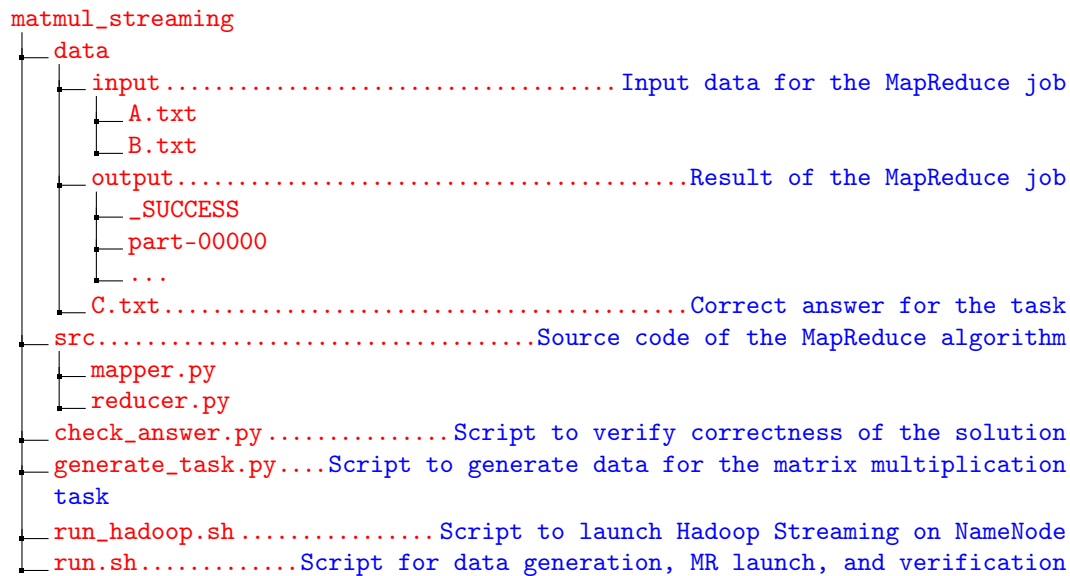


Figure 1: Required project structure

A few important notes that may help with the task:

1. You may use the [Word Count solution](#). In particular, check the `run_hadoop.sh` script.
2. To pass problem dimensions into the mapper and reducer, you can use environment variables. More about setting environment variables for workers can be found [here](#).
3. Flags for changing the number of mappers, reducers, and other fine-tuning options for Hadoop Streaming can be found in the [documentation](#).
4. You may need to determine which file is being processed by a worker at the moment. For this purpose, you can also use [environment variables](#).
5. When printing and saving floating-point numbers, pay attention to the number of digits after the decimal point. About 5 digits are recommended for acceptable precision. Keep in mind that too many digits can significantly slow down MapReduce jobs due to increased data volume.
6. To copy data into and out of the container, you can use the special [docker commands](#).
7. To run programs inside the container, you can also use the special [docker commands](#).

## 2 Recommender Systems (60%)

### 2.1 Introduction

Today, Recommender Systems are found everywhere. On an online store, you may see blocks with “similar products,” on a news site — “related articles” or “stories you may be interested in,” and on a movie rental site — “similar movies” or “recommended for you” sections. We will focus on movie recommendations, but the same principles apply to other domains.

The goal of a Recommender System is to find a small set of movies (`Item`) that are most likely to interest a particular user (`User`), using information about their previous activity and movie characteristics.

A well-known example is the `Netflix` competition launched in 2006, which challenged participants to predict user ratings for movies (on a scale from 1 to 5) based on a known subset of ratings. The winner was the team that improved the `RMSE` on the test dataset by 10% compared to Netflix’s internal solution. Many new methods emerged during the competition.

Typically, the dataset consists of triples  $(u, i, r_{u,i})$ , where  $u$  is the user,  $i$  is the movie, and  $r_{u,i}$  is the rating. We assume that ratings are normalized to the interval  $[0, 1]$ .

## 2.2 Neighborhood Approach in Collaborative Filtering

Given a user-item rating matrix, we can define the `adjusted cosine similarity` measure of similarity between items  $i$  and  $j$  as vectors in user space:

$$\text{sim}(i, j) = \frac{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\sqrt{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_u)^2} \sqrt{\sum_{u \in U_{i,j}} (r_{u,j} - \bar{r}_u)^2}} \quad (1)$$

where  $U_{i,j}$  is the set of users who rated both movies  $i$  and  $j$ , and  $\bar{r}_u$  is the average rating of user  $u$ .

Predicted ratings for unknown movies are computed as follows:

$$\hat{r}_{u,i} = \frac{\sum_{j \in I_u} \text{sim}(i, j) r_{u,j}}{\sum_{j \in I_u} \text{sim}(i, j)} \quad (2)$$

where  $I_u$  is the set of movies rated by user  $u$ . This approach is called `item-oriented`. Note that  $\text{sim}(i, j) \in [-1, 1]$ . This can lead to division by zero or  $\hat{r}_{u,i}$  values outside the range  $[0, 1]$ . To prevent this, we set all negative  $\text{sim}(i, j)$  values to zero.

## 2.3 Assignment Description

In this assignment, you are required to implement collaborative filtering using formulas 1, 2 with the `MapReduce` framework. Your program should take as input a list of triples  $(u, i, r_{u,i})$  and a list mapping movie IDs to movie titles, and output for each user the **top 100** movies with the highest predicted ratings.

When computing recommendations, consider only movies that the user has not yet rated. Recommendations should be sorted in descending order of predicted rating. If two predictions are equal, the movie with the lexicographically smaller title should come first.

Each line in the prediction output file must be formatted as follows:

`<user_id>@<rating_1>%<film_name_1>@...@<rating_100>%<film_name_100>`

Figure 2: Output data format

The dataset to use is [MovieLens](#). Use the `small` version. The output of your program on this dataset should be submitted with your assignment (folder `data/output/final`).

You must also provide a detailed description of your solution in the file `description.md/html/pdf`. Specifically:

1. A description of each stage of your program and each map-reduce job.
2. Time and memory complexity for each mapper and reducer, using the following notations:  $U$  — total number of users,  $I$  — total number of movies,  $M$  — number of mappers,  $R$  — number of reducers,  $\alpha$  — the average fraction of movies rated by one user (equivalently, the average fraction of users who rated one movie, and the ratio of known ratings to total possible  $UI$  ratings).
3. Total runtime of your program.
4. Solutions to bonus tasks (if completed).

## 2.4 Technical Implementation Details

Please note the following points that will help you complete the task successfully:

- [Example of running a Hadoop Streaming program on a cluster](#).
- To use custom separators, use the following options:

```
-D stream.num.map.output.key.fields=<number_of_fields_for_key>
-D stream.map.output.field.separator=<custom_separator>
-D stream.reduce.input.field.separator=<custom_separator>
-D mapreduce.map.output.key.field.separator=<custom_separator>
```

- For implementing secondary sorting, the following options may be useful:

```
-D mapreduce.partition.keycomparator.options=<sort_options>
-D mapreduce.job.output.key.comparator.class= \
    org.apache.hadoop.mapred.lib.KeyFieldBasedComparator
-D mapreduce.partition.keypartitioner.options=<partition_options>
-partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

- To manage memory allocated to containers, use:

```
-D mapreduce.map.memory.mb=<physical_memory_for_each_mapper>
-D mapreduce.reduce.memory.mb=<physical_memory_for_each_reducer>
-D mapreduce.map.java.opts=-Xmx<heap_size_for_each_mapper>m
-D mapreduce.reduce.java.opts=-Xmx<heap_size_for_each_reducer>m
-D yarn.nodemanager.vmem-pmem-ratio=<virtual_to_physical_memory_ratio>
```

- For details on these options and other parameters, see the [Hadoop Streaming documentation](#).
- Note that  $\text{sim}(i, i) = 1 \forall i \in \{1, \dots, I\}$ . Also, ensure there is no division by zero in formulas 1 and 2 by adding a small constant to the denominator (e.g.,  $10^{-9}$ ).
- Use exactly 3 digits after the decimal point when transferring floating-point numbers between workers.
- In `movies.csv`, movie titles may contain commas, so use Python's `csv.reader` to correctly split lines in this file.
- The task can be implemented so that total runtime is about  $\approx 30$  minutes. If your program takes significantly longer, it is likely non-optimal. Non-optimal implementations will be heavily penalized.

## 2.5 Submission Format

You must use Docker containers to run the Hadoop cluster. See the deployment guide in [this repository](#). Other submission formats are allowed only with instructor approval.

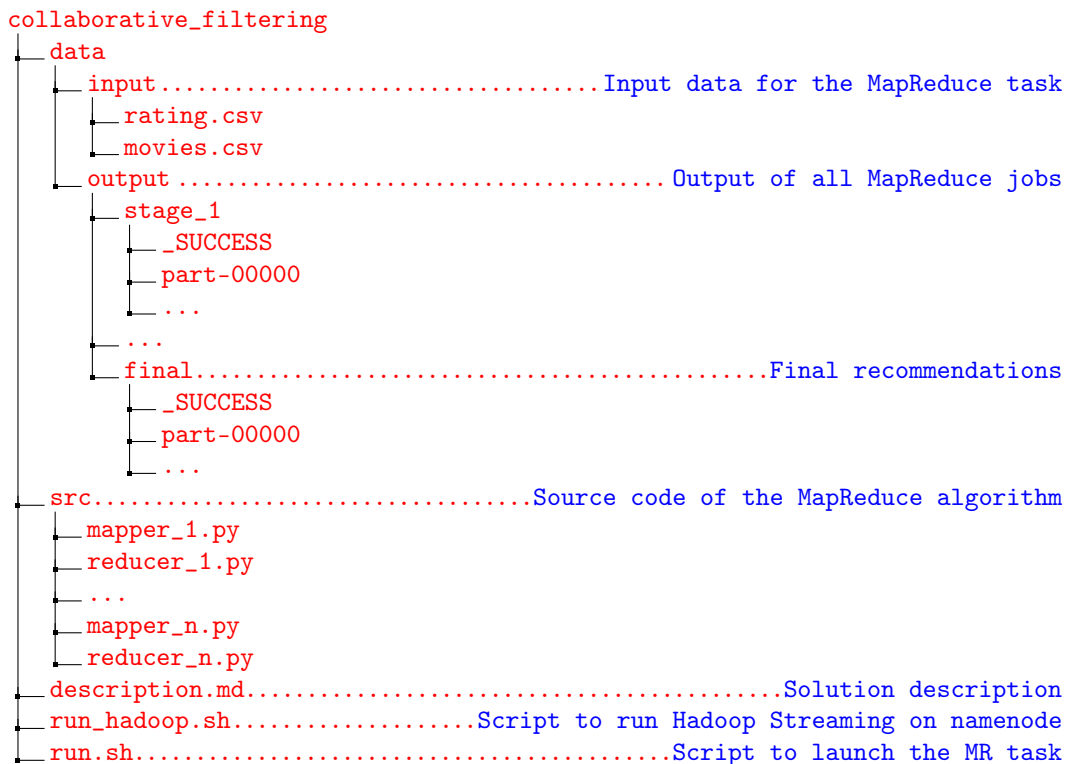


Figure 3: Required project structure

As a result, the project must exactly match the structure shown in Figure 3. Submit an archive of this structure named `<FullName>_task_02.2.zip`.

Details of the submission format:

- The `run.sh` script must provide full task execution: from uploading data into the docker container and running `run_hadoop.sh` to copying the results back from the container. Running the assignment must require only executing this script.
- The `description.md` file contains your solution description. All formulas must be typeset using  $\text{\LaTeX}$ . You may use `Markdown` syntax for formatting. A `Jupyter Notebook` saved as `HTML` is also acceptable.
- The `src` folder contains code used in the assignment. For each map-reduce stage, files must be named `mapper_<stage_n>.py`, `reducer_<stage_n>.py`. Any additional scripts (e.g., combiners) must include the corresponding stage number.
- The `data/input` folder contains the input files (`ratings.csv`, `movies.csv`). During evaluation, these input files will be placed in this directory.
- After running `run.sh`, the `data/output` folder must contain all program outputs. Specifically, inside `data/output/stage_<stage_n>` should be the reducer output (or mapper output in case of a map-only task). The `final` folder must contain the final recommendation list. You may store any intermediate results in `data/output`, but only the contents of `data/output/final` need to be submitted.
- Processing data outside HDFS for use in Replicated Join is allowed.
- Using map-reduce stages with only one reducer is prohibited.
- Full marks will be awarded only if `run.sh` runs successfully on different input datasets.

## 2.6 Bonus Tasks

### 2.7 Analysis of the Obtained Solution (+20%)

Complete the following:

1. Implement a data generation script that produces realistic synthetic data for various  $U, I, \alpha$ .
2. Measure runtime of each stage of your program for different values of  $U, I, \alpha, M, R$ . Be sure to include tests on a dataset larger than the `small` dataset.
3. Plot runtime dependencies on various parameters.
4. Demonstrate, using your experimental data, that your solution's asymptotic complexity matches the theoretical one (e.g., use `log-log` plots). If discrepancies occur, explain why.

### 2.8 Framework Usage (+10%)

Implement your program using the `mrjob` or `Pydoop` framework.

### 2.9 Running on Large Data (+10%)

Run your program on the `large` version of the MovieLens dataset. Attach the resulting output to your solution.